



TRABAJO DE SISTEMAS ELECTRÓNICOS **DIGITALES**

CURSO 2025/2026

TRABAJO MICROPROCESADORES

JUEGO SNAKE



NOMBRE Y APELLIDOS: Laura Lucía Hernández Campos (55289)

Manuel Sánchez Francés (56089)

Pablo García Velasco (56382)

TUTOR: Giuseppe

GRUPO: 31

ÍNDICE

1. Introducción y Descripción.....	3
2. Desarrollo.....	3
3. Diagrama de estados.....	4
4. Diagrama de bloques.....	5
4.1. Bloque de entrada (Sensores).....	5
4.2. Unidad Central de Procesamiento (MCU).....	6
4.3. Bloque de salida (Actuadores e interfaz).....	6
5. Sensores.....	7
5.1. Joystick analógico (Control direccional).....	7
5.2. Botón de usuario (Control de flujo).....	8
6. Comunicación serie (I2C/SPI).....	9
6.1. Protocolo I2C (Inter-Integrated Circuit).....	9
6.2. Protocolo SPI (Serial Peripheral Interface) - Alternativa teórica.....	10
7. Funcionamiento del microcontrolador.....	11
7.1. Gestión del tiempo y dinámica del juego (TIM2).....	11
7.2. Generación de señales para actuadores (PWM - TIM3).....	11
7.3. Adquisición de datos analógicos (ADC1).....	12
7.4. Coordinación mediante Máquina de Estados (FSM).....	12
8. Dificultades con el código.....	12
8.1. Modularización y arquitectura del software.....	13
8.2. Sincronización y gestión de tiempos (conurrencia).....	13
8.3. Abstracción entre lógica y hardware visual.....	14
9. Resultados y validación del sistema.....	14
9.1. Integración hardware-software y concurrencia.....	15
9.2. Dinámica de juego y progresión de dificultad.....	15
9.3. Robustez y persistencia de datos.....	15
9.4. Cumplimiento de objetivos académicos.....	15
10. Síntesis e implementación.....	16
10.1. Resumen del sistema.....	16
10.2. Especificaciones de hardware.....	16
10.3. Especificaciones de software y arquitectura.....	16
10.4. Mapa de recursos internos del microcontrolador (MCU).....	17
11. Enlace al código fuente.....	18

1. Introducción y Descripción

El objetivo de este proyecto es el diseño y desarrollo de una consola de videojuegos interactiva basada en el microcontrolador STM32F407G-DISC1. El sistema integra diversos periféricos de entrada y salida para ofrecer una experiencia de juego electrónica completa, alojada en un chasis de cartón.

El juego implementado es el clásico Snake, donde el usuario controla una serpiente mediante un joystick analógico. La serpiente debe recolectar elementos generados aleatoriamente para aumentar su longitud y puntuación. El sistema, basado en una Máquina de Estados, gestiona la lógica de movimiento, detecta colisiones y proporciona retroalimentación visual mediante una pantalla LCD y sonora a través de señales PWM, garantizando una respuesta en tiempo real.



2. Desarrollo

El proceso de desarrollo se ha estructurado en cinco fases clave para garantizar un sistema robusto y escalable:

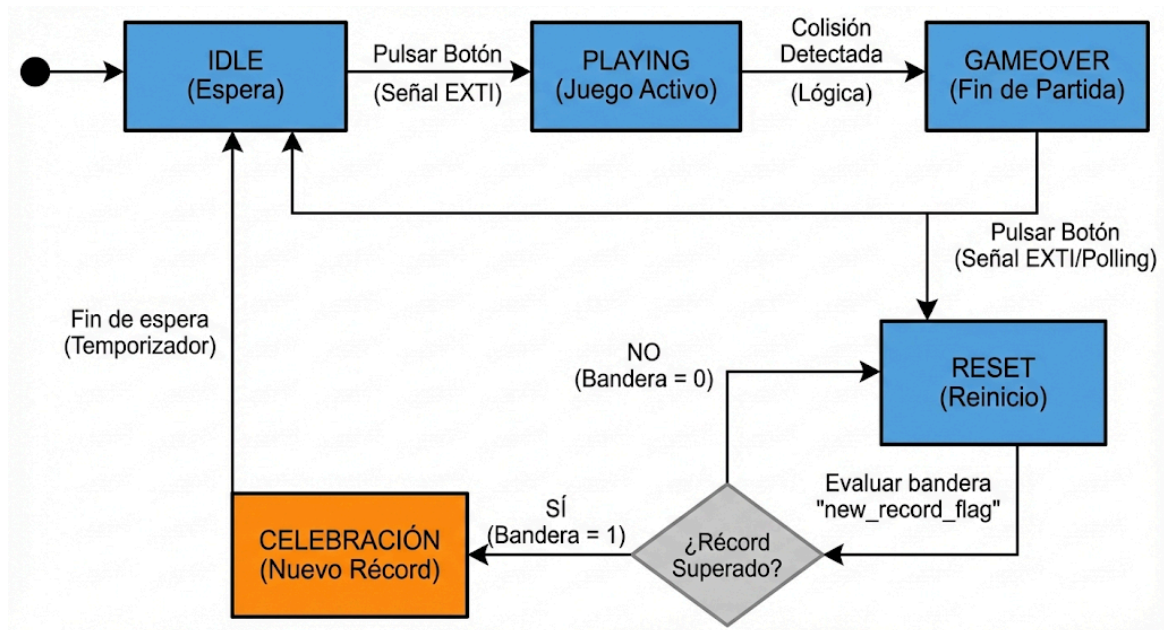
- **Diseño conceptual y lógica de juego:** Definición de la mecánica del Snake en tiempo real. Se diseñó la lógica de crecimiento, la generación aleatoria de comida y las condiciones de colisión, estableciendo un sistema de dificultad progresiva basado en el incremento de velocidad.
- **Arquitectura electrónica y periféricos:** Selección y configuración de componentes utilizando diferentes protocolos de comunicación:
 - Joystick: Entrada analógica mediante el convertidor ADC.
 - Pantalla LCD 16x2: Salida visual mediante el protocolo I2C.
 - Zumbador: Generación de tonos mediante señales PWM.
- **Implementación del hardware y maquetación:** Montaje del circuito en placa de prototipado y posterior diseño de un chasis de cartón. Esta fase incluyó la gestión de conexiones para asegurar la estabilidad de las señales y la accesibilidad de los controles.
- **Desarrollo de software modular:** Programación en lenguaje C siguiendo un esquema de Máquina de Estados Finita (FSM). Se aplicó una estricta separación de responsabilidades en archivos .c y .h (lógica de juego, drivers de pantalla, gestión de sonido y FSM).

- **Integración, depuración y validación:** Pruebas de integración para sincronizar el refresco de pantalla con los tiempos del sistema. Se realizó un proceso de depuración crítico para optimizar el uso de las interrupciones (Callbacks) y evitar bloqueos en el bucle principal durante la generación de sonidos.

3. Diagrama de estados

La lógica del sistema se articula a través de una Máquina de Estados Finita que gestiona no solo el juego, sino también la experiencia de usuario y la retroalimentación de logros:

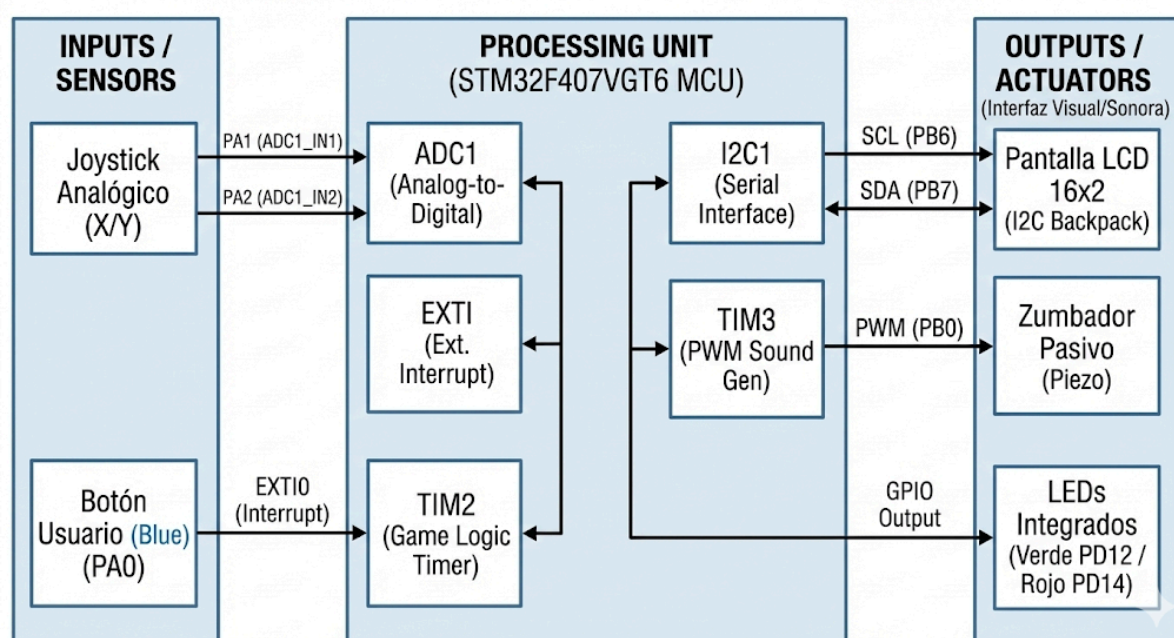
- **IDLE (Espera inicio / High score):** Estado inicial y de reposo. Además de la bienvenida, el sistema consulta la variable global high_score. Si existe un récord previo, se muestra dinámicamente en la segunda línea del LCD. Se mantiene en espera activa hasta que una interrupción externa (EXTI0) inicia la partida.
- **PLAYING (Juego activo):** Durante este estado, el sistema compara en tiempo real la puntuación actual con el récord almacenado. Si el jugador supera la marca, se activa una "bandera" lógica (new_record_flag) que condicionará las transiciones futuras.
- **GAMEOVER (Fin de partida):** Estado de detención. Se deshabilitan los periféricos de movimiento y se muestra el mensaje de fin de juego, permitiendo al usuario asimilar su puntuación antes de proceder al reset.
- **RESET / CELEBRACIÓN (Transición y recompensas):** Es el estado donde se gestiona la lógica de victoria. Al pulsar el botón desde Game Over, el sistema evalúa la bandera de récord:
 - Si hay récord: Se activa una pantalla especial de "NEW RECORD!" durante 5 segundos, acompañada de una melodía de celebración generada por PWM.
 - Si no hay récord: Se muestra el resumen de puntuación estándar. Tras esta fase, el sistema reinicializa las coordenadas y variables de la serpiente, retornando al estado IDLE con la tabla de récords actualizada.



4. Diagrama de bloques

El sistema electrónico diseñado para el juego "Snake" se articula en torno al microcontrolador STM32F407G-DISC1, que actúa como unidad central de procesamiento.

La arquitectura del hardware se divide en tres bloques funcionales principales: bloque de entrada (sensores), bloque de procesamiento y bloque de salida (actuadores e interfaz visual).



A continuación, se describe cada uno de estos bloques:

4.1. Bloque de entrada (Sensores)

Este bloque se encarga de capturar la interacción del usuario y traducirla en magnitudes digitales que el procesador puede gestionar de forma lógica:

- **Joystick analógico (Control direccional):** Basado en dos potenciómetros internos que generan tensiones variables en función de la posición de la palanca. Estas señales analógicas se inyectan en los pines PA1 (Eje X) y PA2 (Eje Y), donde el periférico ADC1 las digitaliza. El software traduce estos valores en comandos de dirección (Arriba, Abajo, Izquierda, Derecha) para la serpiente.
- **Botón de usuario (Gestor de eventos):** Se utiliza el pulsador azul integrado en la placa (pin PA0). Está configurado mediante una línea de interrupción externa (EXTI0), lo que permite que las transiciones entre estados (como iniciar la partida o salir del Game Over) se ejecuten de forma asíncrona e inmediata, sin necesidad de consultar el estado del botón constantemente en el bucle principal (polling).

4.2. Unidad Central de Procesamiento (MCU)

El núcleo del sistema es el microcontrolador STM32F407VGT6, basado en la arquitectura ARM Cortex-M4. Este dispositivo actúa como el orquestador central, ejecutando el firmware del juego y gestionando de forma concurrente los periféricos internos necesarios para la lógica del sistema:

- **ADC1 (Analog-to-Digital Converter):** Configurado para muestrear los canales correspondientes a los ejes X e Y del joystick. La digitalización de estas señales permite al software mapear las variaciones de tensión en comandos direccionales precisos, procesando los valores mediante un umbral de histéresis para evitar movimientos erráticos.
- **TIM2 (Motor de tiempo y velocidad):** Es el temporizador de propósito general encargado de generar la base de tiempos del juego. Mediante interrupciones periódicas (Update Interrupts), dicta el "tic" de movimiento de la serpiente. Su configuración es dinámica, permitiendo reducir el periodo de desbordamiento para incrementar la dificultad de forma progresiva.
- **TIM3 (Generador de frecuencias PWM):** Utilizado para el control del zumbador pasivo. Genera señales de Modulación por Ancho de Pulso (PWM) donde se varía la frecuencia para producir diferentes tonos musicales, permitiendo la retroalimentación sonora sin bloquear la ejecución del código principal.
- **I2C1 (Comunicación serie síncrona):** Actúa como maestro en la comunicación con el driver de la pantalla LCD. Este protocolo permite el envío de tramas de datos complejas utilizando únicamente dos líneas (SDA y

SCL), optimizando el diseño del hardware y liberando pines de entrada/salida general (GPIO).

- **EXTI (Gestor de interrupciones externas):** Configurado para detectar flancos de subida en el pin PA0. Al estar asociado al botón físico, garantiza que los cambios de estado en la FSM (como el inicio de partida o el reinicio tras la muerte) se procesen de forma prioritaria y asíncrona, mejorando la latencia de respuesta del sistema.

4.3. Bloque de salida (Actuadores e interfaz)

Este bloque proporciona retroalimentación visual y sonora al jugador en tiempo real, traduciendo el estado lógico procesado por la MCU en estímulos físicos:

- **Interfaz visual (Pantalla LCD 16x2):** Actúa como el canal principal de comunicación con el usuario. A través del protocolo I2C (pines PB6/SCL y PB7/SDA), se envían los datos necesarios para renderizar el tablero, la posición de la comida y la puntuación dinámica. El uso de este bus serie permite una gestión eficiente de los recursos del microcontrolador al reducir drásticamente el número de líneas de datos necesarias para controlar el controlador HD44780 del LCD.
- **Interfaz sonora (Zumbador piezoeléctrico):** Actuador acústico conectado al pin PB0. Su funcionamiento se basa en la señal PWM generada por el temporizador TIM3, lo que permite modular la frecuencia de salida para producir tonos diferenciados. El sistema emite bips cortos de alta frecuencia al recolectar comida y tonos graves sostenidos en caso de colisión, mejorando la inmersión y el game feel.
- **Señalización de estado (LEDs integrados):** Se utilizan los diodos de propósito general integrados en la placa Discovery como indicadores de estado de alta prioridad:
- **LED verde (PD12):** Permanece activo exclusivamente durante el estado PLAYING, indicando que el motor de juego y los timers están operativos.
- **LED rojo (PD14):** Se activa de forma fija al detectar una colisión, reforzando visualmente el estado de GAMEOVER.

5. Sensores

El sistema de juego interactivo requiere detectar las intenciones del usuario en tiempo real. Para ello, se han implementado dos tipos de transductores de entrada que convierten acciones físicas en señales eléctricas interpretables por el microcontrolador STM32F407.

5.1. Joystick analógico (Control direccional)

El control de movimiento de la serpiente se realiza mediante un módulo de joystick analógico de dos ejes.

Principio de funcionamiento físico: Este sensor consta internamente de dos potenciómetros de 10 k Ω montados perpendicularmente entre sí. Al mover la palanca, se desplaza el cursor de estos potenciómetros, variando la resistencia en cada eje.

Interfaz eléctrica y conexión: El módulo se alimenta con los 3.3V de la placa Discovery. Dependiendo de la posición de la palanca, los pines de salida de señal (VRX y VRY) proporcionan una tensión analógica variable entre 0V (GND) y 3.3V (VCC). En su posición central de reposo, la tensión de salida es aproximadamente la mitad de la alimentación (1.65V).

Las conexiones al microcontrolador son las siguientes:

- Eje X (VRX): Conectado al pin PA1.
- Eje Y (VRY): Conectado al pin PA2.

Procesamiento en el microcontrolador (ADC): El STM32F407 utiliza su periférico interno ADC1 (Conversor Analógico-Digital) con una resolución de 12 bits para leer estas tensiones. El ADC muestrea la señal analógica y la convierte en un valor digital entero entre 0 y 4095.

La lógica del software (Leer_Joystick_ADC en main.c) interpreta estos valores:

- Un valor cercano a 2048 indica que el joystick está centrado (sin movimiento).
- Un valor bajo en un eje (por ejemplo, < 1000) indica un desplazamiento hacia un extremo (izquierda o arriba).
- Un valor alto en un eje (por ejemplo, > 3000) indica un desplazamiento hacia el extremo opuesto (derecha o abajo).

Esta lectura se realiza cíclicamente en el bucle principal para actualizar la variable de dirección de la serpiente antes de su siguiente movimiento.

5.2. Botón de usuario (Control de flujo)

Para las acciones críticas de inicio y reinicio del sistema, se utiliza el pulsador azul integrado en la placa STM32F407G-DISC1, el cual permite una gestión directa del flujo de la Máquina de Estados.

- **Principio de funcionamiento físico y eléctrico:** Se trata de un pulsador momentáneo normalmente abierto conectado al pin PA0. El hardware está configurado en modo Active-High (activo a nivel alto):

- Estado de reposo: Una resistencia de pull-down interna garantiza que el pin se mantenga en un nivel lógico bajo (0V), evitando lecturas erráticas por ruido.
 - Estado de activación: Al presionar el botón, se cierra el contacto mecánico, aplicando la tensión de alimentación (3V) al pin PA0 y generando un nivel lógico alto.
- **Gestión por interrupciones externas (EXTI)**: A diferencia del joystick, cuya lectura es cíclica, el estado del botón se gestiona mediante el controlador de Interrupciones Externas (EXTI). Esta elección de diseño es fundamental por dos razones:
- Eficiencia: Se evita el polling (sondeo continuo), liberando ciclos de CPU para la lógica del juego y el refresco del LCD.
 - Inmediatez: El sistema responde al usuario de forma asíncrona y prioritaria.
- **Procesamiento en la CPU (NVIC y Callback)**: El pin PA0 está configurado para disparar un evento ante un flanco de subida (rising edge). Cuando se detecta la transición:
- El controlador NVIC (Nested Vectored Interrupt Controller) suspende momentáneamente la ejecución del bucle principal.
 - Se ejecuta la rutina de servicio o callback específica.
 - Dentro de esta rutina, se actualiza la variable de estado de la FSM (por ejemplo, transicionando de IDLE a PLAYING o de GAMEOVER a RESET).

Esta arquitectura garantiza que el sistema detecte la pulsación de forma instantánea, incluso si el microcontrolador está realizando operaciones intensivas de escritura en el bus I2C o cálculos de colisiones en ese preciso instante.

6. Comunicación serie (I2C/SPI)

Para la visualización del estado del juego, el sistema requiere transmitir datos desde el microcontrolador a un módulo de pantalla externo. Dado que el número de pines GPIO disponibles es limitado y se busca simplificar el cableado del prototipo, se opta por utilizar protocolos de comunicación serie sincrónicos.

Aunque el diseño final implementa el protocolo I2C para el manejo de la pantalla LCD, el sistema se ha diseñado teniendo en cuenta la alternativa de SPI para

posibles expansiones (como matrices de LEDs), cubriendo así los dos buses de comunicación más habituales en electrónica digital.

6.1. Protocolo I2C (Inter-Integrated Circuit)

El proyecto utiliza el bus I2C para controlar la pantalla LCD 16x2 a través de un adaptador basado en el chip expensor de E/S PCF8574.

Principio de funcionamiento: I2C es un protocolo de comunicación serie sincrónico, half-duplex y de arquitectura maestro-esclavo. Su principal ventaja es que permite conectar múltiples dispositivos utilizando únicamente dos líneas de señal bidireccionales (más alimentación):

SDA (Serial Data Line): Línea por donde se transmiten los datos bit a bit.

SCL (Serial Clock Line): Línea de reloj que sincroniza la transmisión de datos, generada siempre por el maestro.

Ambas líneas son de "drenador abierto" (open-drain), lo que requiere resistencias de pull-up para mantener el nivel alto cuando ningún dispositivo está transmitiendo.

Implementación en el proyecto: El microcontrolador STM32F407 actúa como Maestro, iniciando toda comunicación, y el módulo adaptador de la LCD actúa como Esclavo.

Se utiliza el periférico interno I2C1 del STM32, configurado en los siguientes pines:

- SCL: Conectado al pin PB6.
- SDA: Conectado al pin PB7.

El proceso de comunicación para actualizar la pantalla es el siguiente:

1. El MCU genera una condición de START.
2. Envía la dirección física (address) del chip esclavo PCF8574 en el bus (generalmente 0x27 o 0x3F).
3. Tras recibir el bit de confirmación (ACK) del esclavo, el MCU envía bytes de datos.
4. Estos datos no van directamente al cristal líquido, sino al registro del chip PCF8574, que a su vez activa sus pines de salida paralela conectados a la interfaz cruda de la pantalla LCD (pines RS, RW, EN, D4-D7).
5. El MCU genera una condición de STOP para liberar el bus.

Este método permite controlar una pantalla que nativamente requeriría más de 6 pines digitales utilizando solo 2 pines del microcontrolador.

6.2. Protocolo SPI (Serial Peripheral Interface) - Alternativa teórica

Aunque no se implementó en el prototipo final de LCD, el protocolo SPI se consideró como alternativa para el uso de visualizadores más rápidos, como matrices de puntos LED (ej. controladores MAX7219), habituales en juegos tipo Snake.

- **Características principales:** SPI es un protocolo síncrono que opera en modo full-duplex (puede transmitir y recibir simultáneamente), permitiendo velocidades de transmisión significativamente más altas que I2C. A diferencia de I2C, que usa direcciones, SPI utiliza una línea de selección de chip física para cada esclavo.

Requiere cuatro líneas de señal:

1. SCLK (Serial Clock): Reloj generado por el maestro.
2. MOSI (Master Output Slave Input): Datos del maestro al esclavo.
3. MISO (Master Input Slave Output): Datos del esclavo al maestro (no usada si solo se envía vídeo a una pantalla).
4. CS/SS (Chip Select/Slave Select): Línea activa a nivel bajo para habilitar un esclavo específico.

El conocimiento de ambos protocolos es esencial para la selección adecuada de periféricos en función de las necesidades de velocidad (SPI) frente a la simplicidad de cableado (I2C).

7. Funcionamiento del microcontrolador

El núcleo del sistema es el microcontrolador STM32F407VGT6, basado en la arquitectura ARM Cortex-M4 de 32 bits con unidad de punto flotante (FPU). Su función es actuar como un controlador que gestiona múltiples periféricos hardware en paralelo para garantizar la respuesta en tiempo real del sistema.

El funcionamiento interno del microcontrolador en este proyecto se basa en la coordinación de los siguientes bloques funcionales clave:

7.1. Gestión del tiempo y dinámica del juego (TIM2)

Un aspecto crítico en el desarrollo de videojuegos es mantener una velocidad de actualización constante, independientemente de la carga de procesamiento. En lugar de utilizar bucles de retardo por software (HAL_Delay), que bloquearían la CPU, se emplea un Temporizador de Hardware (TIM2).

El TIM2 está configurado como una base de tiempos que genera una interrupción periódica. Este temporizador funciona de forma autónoma a la CPU, contando ciclos

de reloj del sistema. Cuando el contador alcanza el valor predefinido en su registro de auto-recarga (ARR), dispara una petición de interrupción.

El controlador de interrupciones (NVIC) detiene momentáneamente el bucle principal del programa y salta a la Rutina de Servicio de Interrupción (ISR) del TIM2. Es dentro de esta rutina donde se ejecuta la lógica crítica del juego: calcular la nueva posición de la cabeza de la serpiente y desplazar su cuerpo. Esto garantiza que el "ritmo" del juego sea preciso y estable.

7.2. Generación de señales para actuadores (PWM - TIM3)

Para generar la retroalimentación sonora a través del zumbador pasivo, el microcontrolador no se limita a encender y apagar un pin digital. Se utiliza el temporizador TIM3 configurado en modo Modulación por Ancho de Pulso (PWM).

El hardware del temporizador genera una onda cuadrada en el pin de salida PB0 sin intervención constante de la CPU. El software modifica "al vuelo" los registros de frecuencia (ARR) y ciclo de trabajo (CCR) del temporizador para variar el tono de la señal de audio. Esto permite generar notas musicales distintas para indicar eventos diferentes (sonido agudo al comer, sonido grave al chocar) de manera eficiente.

7.3. Adquisición de datos analógicos (ADC1)

El microcontrolador interactúa con el mundo analógico del joystick mediante su Conversor Analógico-Digital (ADC1) interno de 12 bits.

El ADC se configura para muestrear secuencialmente los canales conectados a los ejes X e Y. Mediante el método de aproximaciones sucesivas, convierte la tensión de entrada (entre 0 V y 3 V) en un valor digital entero (0 a 4095). El software principal sondea (hace polling) estos valores convertidos en cada ciclo del bucle principal para actualizar la dirección de movimiento deseada por el jugador antes de que ocurra el siguiente "tic" del temporizador de movimiento.

7.4. Coordinación mediante Máquina de Estados (FSM)

El bucle principal del programa (main while(1)) no ejecuta una secuencia lineal de instrucciones de juego. En su lugar, alberga la implementación de una Máquina de Estados Finitos (FSM).

En cada iteración, el microcontrolador evalúa en qué estado se encuentra el sistema (IDLE, PLAYING, GAMEOVER, RESET) y ejecuta únicamente el código correspondiente a ese estado.

La transición entre estados es provocada por eventos externos (detectados por interrupciones, como el botón PA0) o internos (detectados por la lógica del juego,

como una colisión). Esta arquitectura permite un comportamiento del sistema predecible, robusto y fácil de depurar.

8. Dificultades con el código

El desarrollo del firmware para este sistema embebido presentó desafíos significativos que van más allá de la programación secuencial convencional en C. La necesidad de gestionar múltiples periféricos hardware en tiempo real, manteniendo una estructura de código mantenible y modular, supuso las principales dificultades del proyecto.

A continuación, se detallan los principales obstáculos encontrados y las soluciones técnicas implementadas:

8.1. Modularización y arquitectura del software

El desafío: Uno de los requisitos fundamentales de la asignatura era evitar la creación de un código monolítico dentro del archivo main.c.

El principal reto fue realizar la transición de un código inicial centralizado a una arquitectura modular robusta, separando la lógica en diferentes unidades de compilación (fsm.c, snake.c, display.c, sound.c).

Esto generó problemas iniciales de compilación y enlace, como errores de "definición múltiple" de variables globales al incluirlas en cabeceras, o dificultades para gestionar las dependencias cruzadas entre módulos (por ejemplo, la máquina de estados necesita conocer la estructura de la serpiente, y la pantalla necesita conocer el estado del juego).

La solución: Se implementó una estricta disciplina de programación en C para sistemas embebidos:

- Separación Interfaz/Implementación: Uso riguroso de archivos de cabecera (.h) para declaraciones de funciones públicas y tipos de datos, y archivos fuente (.c) para las definiciones y variables privadas.
- Guardas de Inclusión: Uso de directivas de preprocesador (#ifndef, #define, #endif) en todos los archivos .h para evitar inclusiones cíclicas.
- Gestión de Variables Globales: Uso de la palabra clave extern en los .h para compartir variables de estado críticas (como myGame o mySnake) sin provocar errores de redefinición.

8.2. Sincronización y gestión de tiempos (conurrencia)

El desafío: Un videojuego requiere una gestión precisa del tiempo. Un enfoque inicial e ingenuo para controlar la velocidad de la serpiente fue utilizar retardos bloqueantes (HAL_Delay) dentro del bucle principal. Esto provocaba que el microcontrolador quedara "congelado" durante esos retardos, dejando de responder a las pulsaciones del botón de usuario o perdiendo lecturas del joystick, resultando en una experiencia de juego lenta y poco reactiva.

La solución: Se cambió el paradigma de programación secuencial por un diseño orientado a eventos e interrupciones:

- Se delegó la base de tiempos del juego al hardware del temporizador TIM2.
- El movimiento de la serpiente se ejecuta exclusivamente dentro de la rutina de callback de la interrupción del temporizador (HAL_TIM_PeriodElapsedCallback).

Esto libera al bucle principal (while(1)), que ahora se dedica a tareas no bloqueantes: ejecutar la máquina de estados y sondear el ADC del joystick, garantizando que el sistema siempre responda a las entradas.

8.3. Abstracción entre lógica y hardware visual

El desafío: La lógica interna del juego (snake.c) trabaja con un sistema de coordenadas cartesianas abstracto (ej. cabeza en x=5, y=2). El reto consistió en traducir eficientemente este modelo matemático a señales físicas I2C comprensibles por el controlador de la pantalla LCD 16x2 para representar el tablero.

La solución: Se creó una capa de abstracción de hardware (HAL) específica en el módulo display.c. Las funciones de este módulo actúan como "traductores": reciben el estado abstracto del juego (estructuras Snake_t y Food_t) y calculan qué comandos de bajo nivel (mover cursor, escribir carácter 'X' u 'O') deben enviarse por el bus I2C para reflejar ese estado visualmente, desacoplando la lógica del juego del hardware de visualización específico.

9. Resultados y validación del sistema

Como resultado final, se ha implementado con éxito un prototipo funcional de consola interactiva basado en el STM32F407G-DISC1. El sistema no solo cumple con las especificaciones iniciales, sino que integra funciones avanzadas de control y experiencia de usuario en tiempo real.



9.1. Integración hardware-software y concurrencia

El prototipo demuestra una coordinación efectiva entre la lógica de software y los periféricos físicos. Gracias al uso de interrupciones y periféricos autónomos, el microcontrolador gestiona simultáneamente:

- **Adquisición:** Muestreo del joystick (ADC) y detección de eventos (EXTI).
- **Actuación:** Generación de audio (PWM) y señalización de estados (LEDs).
- **Comunicación:** Actualización del LCD mediante el bus I2C. Esta integración se traduce en un sistema con latencia imperceptible, donde el sonido y la imagen están perfectamente sincronizados con las acciones del jugador.

9.2. Dinámica de juego y progresión de dificultad

La implementación basada en el temporizador TIM2 ha permitido una fluidez constante. Como resultado destacado, se logró implementar una curva de dificultad dinámica: conforme el jugador recolecta comida, el firmware reduce el periodo del temporizador, aumentando la velocidad de la serpiente de forma progresiva. Esto demuestra un uso avanzado de la reconfiguración de registros de hardware en tiempo de ejecución.

9.3. Robustez y persistencia de datos

La arquitectura de Máquina de Estados Finitos (FSM) garantiza que el sistema sea determinista. Un resultado crítico es la implementación de la gestión de récords (High Score):

- El sistema monitoriza la puntuación máxima durante la sesión.
- En caso de superar el récord, se activa una transición específica de "Celebración" con feedback visual y sonoro exclusivo.
- El sistema permite reinicios limpios mediante el botón de usuario, manteniendo la persistencia del récord mientras la placa esté alimentada.

9.4. Cumplimiento de objetivos académicos

El proyecto valida la adquisición de las competencias fundamentales de la asignatura:

- **Programación modular:** Separación estricta de lógica de juego, drivers de periféricos y gestión de estados en archivos .c y .h.
- **Control de periféricos:** Configuración manual y uso de ADC, PWM, Timers e I2C.
- **Sistemas de tiempo real:** Gestión de prioridades mediante el controlador NVIC para evitar colisiones entre interrupciones.

10. Síntesis e implementación

Este anexo presenta un resumen narrativo de las características técnicas fundamentales, los recursos hardware utilizados y la arquitectura software implementada en el prototipo final del juego "Snake".

10.1. Resumen del sistema

El proyecto consiste en un sistema embebido interactivo en tiempo real que implementa la lógica del clásico juego "Snake". El núcleo del sistema es un microcontrolador de 32 bits encargado de gestionar la lectura de entradas analógicas y digitales, procesar la lógica de juego y generar respuestas visuales y sonoras coordinadas a través de diversos buses de comunicación y señales moduladas.

10.2. Especificaciones de hardware

El corazón de la implementación física es la placa de desarrollo STM32F407G-DISC1, basada en el procesador ARM Cortex-M4.

Para la interacción con el usuario, el sistema incorpora dos tipos de entradas. El control direccional se realiza mediante un módulo de joystick analógico (tipo KY-023) de dos ejes, cuyas salidas están conectadas a los pines físicos PA1 (Eje X) y PA2 (Eje Y). El control de flujo del juego (inicio y reinicio) se gestiona a través del pulsador de usuario azul integrado en la propia placa, conectado al pin PA0.

El sistema cuenta con múltiples actuadores de salida. La interfaz visual principal es una pantalla LCD de 16x2 caracteres equipada con un módulo adaptador I2C, conectada a los pines PB6 (SCL) y PB7 (SDA), utilizada para mostrar el tablero y la puntuación. De forma complementaria, se utilizan los LEDs integrados en la placa Discovery (Verde en PD12 y Rojo en PD14) como indicadores rápidos de estado. Finalmente, la retroalimentación auditiva se genera mediante un zumbador piezoeléctrico pasivo conectado al pin PB0, capaz de emitir tonos de diferente frecuencia.

10.3. Especificaciones de software y arquitectura

Entorno y framework de desarrollo: El firmware del sistema ha sido desarrollado utilizando el lenguaje de programación C estándar (C99) dentro del entorno de desarrollo integrado STM32CubeIDE. El código se apoya en las librerías de controladores STM32 HAL (Hardware Abstraction Layer) para el acceso al hardware de bajo nivel.

Paradigma de ejecución y patrones de diseño: El flujo de la aplicación se rige por un patrón de Máquina de Estados Finitos (FSM). Este enfoque garantiza que el sistema se comporte de forma predecible y robusta ante cualquier evento. La arquitectura es de tipo orientada a eventos y dirigida por interrupciones:

- Ejecución no bloqueante: Se evita estrictamente el uso de retardos por software (HAL_Delay) en el bucle principal.
- Gestión de tiempos: El motor de juego se sincroniza mediante la ISR (Interrupt Service Routine) del temporizador TIM2, asegurando una tasa de refresco constante.
- Sincronización: La comunicación entre las interrupciones hardware (botones, timers) y el bucle principal se realiza mediante banderas lógicas (flags), minimizando la latencia de respuesta del sistema.

Modularidad y organización del código: La estructura del proyecto sigue un diseño estrictamente modular, separando responsabilidades en archivos de cabecera (.h) y fuente (.c). Los módulos principales incluyen:

- main: Inicialización de la capa HAL, configuración de relojes (RCC) y de periféricos.
- fsm: Transiciones entre estados y gestión de la lógica de usuario.
- snake: Implementación del motor físico (movimiento, crecimiento y detección de colisiones).
- display: Capa de abstracción para el controlador LCD I2C y renderizado de caracteres.
- sound: Driver de audio encargado de la modulación de frecuencia mediante registros PWM.

10.4. Mapa de recursos internos del microcontrolador (MCU)

El microcontrolador STM32F407 emplea varios de sus periféricos internos para el funcionamiento del sistema.

Para la gestión del tiempo, se utilizan dos temporizadores: el TIM2, configurado para generar una interrupción periódica que sirve como base de tiempos para el movimiento de la serpiente, y el TIM3, configurado en modo de salida PWM en el canal 3 para generar las señales de audio de frecuencia variable para el zumbador.

La interfaz con el mundo exterior se gestiona mediante interfaces analógicas y digitales. El ADC1 (Conversor Analógico-Digital) se utiliza con una resolución de 12 bits en modo de escaneo para leer secuencialmente los voltajes de los ejes del joystick. La comunicación con la pantalla LCD se realiza mediante la interfaz serie I2C1, configurada en modo Maestro a velocidad estándar (100 kHz).

Por último, la gestión de eventos externos críticos se realiza mediante el controlador de interrupciones externas EXTI0, configurado para detectar flancos de subida en el pin PA0, permitiendo una detección inmediata de la pulsación del botón de usuario. Todas estas interrupciones son gestionadas y priorizadas por el controlador NVIC del núcleo ARM.

11. Enlace al código fuente

Todo el código fuente se encuentra alojado en el siguiente repositorio, demostrando el trabajo continuo del grupo:

URL del repositorio: https://github.com/LauraHCampos/SED_Micros_Grupo31